

DAY 27

Decision Trees in Machine Learning

60-Day Data Science Learning Journey | Muhammad Ahmad

Learning Objectives

Understand Decision Trees, classification and regression, Gini impurity, entropy, information gain, hyperparameters, overfitting, evaluation, feature importance, visualization, and practical prediction.

1. Decision Trees

A Decision Tree is a supervised Machine Learning algorithm that learns a sequence of feature-based decisions. Each split creates branches, and a final leaf produces the prediction. Classification predicts categories such as Pass or Fail. Regression predicts numeric values such as exam marks.

2. Tree Terminology

Root node = first decision. Internal node = decision point. Branch = path after a split. Leaf = final prediction. Depth = number of levels. Split = dividing data using a feature and threshold. Purity = how homogeneous a node is.

3. Gini Impurity

$Gini = 1 - \sum(p_i^2)$. For a binary node containing 80% class A and 20% class B, $Gini = 1 - (0.8^2 + 0.2^2) = 0.32$. A pure node has Gini 0. A balanced binary node has maximum Gini 0.5.

4. Entropy and Information Gain

Entropy is $H = -\sum(p_i \log_2(p_i))$. A pure node has entropy 0, while a balanced binary node has entropy 1. Information Gain = Parent Entropy - Weighted Child Entropy. The tree searches for splits that improve the selected criterion.

5. Important Hyperparameters

`max_depth` limits tree depth. `min_samples_split` controls the minimum samples needed to split a node. `min_samples_leaf` controls the minimum samples in a leaf. `criterion` can use gini or entropy, and newer scikit-learn versions also support `log_loss`. `max_features` limits features considered for a split.

6. Scaling and Overfitting

Decision Trees generally do not require feature scaling because they make threshold-based splits. Very deep trees can memorize training data and overfit. `max_depth`, `min_samples_split`, and `min_samples_leaf` can help control complexity.

7. Complete Classification Project

Student Pass/Fail Prediction using DecisionTreeClassifier

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

data = {
    "study_hours": [1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9, 10, 10, 11],
    "attendance": [50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 87, 90, 91, 93, 95, 96, 98],
    "previous_gpa": [1.8, 2.0, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3.0, 3.0, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7],
    "assignment_score": [35, 40, 42, 45, 48, 52, 55, 58, 62, 65, 68, 70, 74, 76, 80, 82, 85, 88, 92, 95],
    "sleep_hours": [5, 5, 5.5, 5.5, 6, 6, 6, 6.5, 6.5, 7, 7, 7, 7.5, 7.5, 8, 8, 8, 8.5, 8.5],
    "passed": ["Fail", "Fail", "Fail", "Fail", "Fail", "Fail", "Fail", "Fail", "Pass", "Pass", "Pass",
              "Pass", "Pass", "Pass", "Pass", "Pass", "Pass", "Pass", "Pass", "Pass"]
}

df = pd.DataFrame(data)

X = df[["study_hours", "attendance", "previous_gpa", "assignment_score", "sleep_hours"]]
y = df["passed"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_split=4,
    min_samples_leaf=2,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

print("\nFeature Importance:")
for feature, importance in zip(X.columns, model.feature_importances_):
    print(feature, ":", round(importance, 4))

new_student = pd.DataFrame({
    "study_hours": [6],
    "attendance": [82],
    "previous_gpa": [3.0],
    "assignment_score": [72],
    "sleep_hours": [7]
})

prediction = model.predict(new_student)
probability = model.predict_proba(new_student)

print("\nNew Student Prediction:", prediction[0])
print("Class Probabilities:", probability[0])

plt.figure(figsize=(16, 9))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True
)
plt.title("Student Pass/Fail Decision Tree")
plt.tight_layout()
plt.show()
```

8. Line-by-Line Understanding

train_test_split() separates training and test data. **stratify=y** keeps class proportions similar. **DecisionTreeClassifier()** creates the model. **model.fit()** learns rules. **predict()** produces classes. **predict_proba()** returns class probabilities. **confusion_matrix()** compares actual and predicted classes. **classification_report()** gives precision, recall, F1-score and support. **feature_importances_** gives impurity-based importance values. **plot_tree()** visualizes the learned

structure.

9. Decision Tree Regression

```
from sklearn.tree import DecisionTreeRegressor
```

```
model = DecisionTreeRegressor(  
    max_depth=4,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)  
predictions = model.predict(X_test)
```

For regression, the target must be numeric. Compare predictions using MAE, MSE and R^2 .

10. Practice Tasks and Solutions

1. Change `max_depth` from 4 to 2 and compare test performance. Solution: use `DecisionTreeClassifier(max_depth=2, random_state=42)`.
2. Try `max_depth=10` and inspect tree complexity. Solution: train a second model and compare training and test scores.
3. Compare Gini and entropy. Solution: train models with `criterion='gini'` and `criterion='entropy'`.
4. Print feature importance. Solution: loop through `zip(X.columns, model.feature_importances_)`.
5. Build a regression tree for exam marks. Solution: use `DecisionTreeRegressor` and numeric `y`.
6. Compare Linear Regression and Decision Tree Regression with MAE, MSE and R^2 on the same split.

11. Key Takeaways

Decision Trees are easy to understand and can solve classification and regression. They usually do not need scaling. Gini and entropy help select classification splits. Deep trees can overfit. Test-set evaluation is essential. Feature importance helps interpretation but does not prove causation.

12. Next Step

The next stage moves toward ensemble learning, where multiple decision trees can work together for stronger and more robust predictions.